The AI Security Showdown

Tabish Izhar
Otto Friedrich University Bamberg
Bamberg, Germany
tabish.izhar@stud.uni-bamberg.de

Muhammad Usman
Otto Friedrich University Bamberg
Bamberg, Germany
muhammad2.usman@stud.uni-bamberg.de

***Abstract*—The increasing tendency of utilizing large language models (LLMs) for code generation has brought about security concerns regarding the security of AI-generated codes. This paper aims to empirically evaluate the security of 883,140 AI-generated C codes from the FormAI-v2 dataset. The codes were generated by nine different AI models and formally verified. The security differences of the AI models in terms of vulnerability density and the relationship between code complexity and security vulnerabilities have also been evaluated. The results reveal that the security of AI-generated codes varies significantly across different AI models. GPT-4o-mini has the lowest vulnerability density, while Gemma-7B has the highest. Moreover, a weak but statistically significant negative relationship has been found between cyclomatic complexity and the presence of vulnerabilities.**

***Index Terms*—component, formatting, style, styling, insert.**

I. Introduction

The rapid emergence of large language models (LLMs) in coding is transforming the field of modern software development. Artificial intelligence-based coding assistants are becoming a norm for code generation, refactoring, and assisting programmers in completing tasks quickly. They promise a huge productivity gain by automating mundane coding tasks and making software development accessible to more people. As a result, AI-coded software is no longer limited to proof-of-concept demonstrations but is being used in production environments.

While we have all the above advantages, relying even more on AI-generated code brings new challenges, and we can better understand this by looking at software security. We are now in an era where software plays an important role in safety-critical and security-sensitive applications, and we can think of examples in the medical field, finance, and infrastructure. In all these areas, security vulnerabilities can cause consequences such as data breaches, system compromises, and even financial losses. Therefore, ensuring that AI-generated code is secure is not just an abstract concern, but a practical problem that affects both developers and researchers.

As traditional software engineering clearly shows, software quality is really a matter of things like the programmer's skill level, the messiness of the code, and tools available. In other words, more complex code usually means more bugs, and more experienced programmers usually mean more reliable code. But what we want to know is if these same rules apply to code that is not being programmed by a human at a keyboard, but is instead being programmed by a machine, or an AI. Programs created by an AI can be quite different in their structure, abstraction, level of repetition, and control flow, and so on, and so it is difficult to know if these traditional quality metrics really apply to code that is being created by a machine.

The biggest lingering question, of course, is whether these models produce code that possesses similar security characteristics. While large language models are quite different in their design and training, as well as their number of parameters, these differences could potentially influence their security in code generation. If certain models are found to produce code that is more secure, that is an incredibly useful knowledge that could be employed by developers and organizations looking to leverage AI in their coding processes. Conversely, if certain models are more likely to produce insecure code, additional precautions would be necessary in their usage.

Current empirical research is beginning to assess code generated by AI on multiple dimensions: correctness, reliability, and usability. A significant amount of this research focuses on functional correctness, compilation, and performance on typical benchmarking tasks. While these approaches are very informative, they often overlook security-related characteristics. Many current research efforts also rely on heuristic static analysis tools to identify vulnerabilities. While these tools are widely used in practice, they are susceptible to false positives and false negatives, making it more difficult to interpret results.

However, interpreting the results of automated security analysis is also fraught with its own challenges. Security vulnerabilities are typically rare occurrences, and if we misunderstand what the analysis tools are telling us, it is very easy to be misled. One common example of this is the prosecutor's fallacy, where we might confuse the probability of observing a vulnerability with the probability that a discovered problem actually represents a genuine security risk. This has been a recurring theme in empirical software engineering and software testing research. Without normalizing and putting the data into context, simple numbers of vulnerabilities can cloud our perception of software security, for instance, concluding that more warnings from a tool or model necessarily mean that the system is less secure.

In addition to the choice of the most suitable AI model, code complexity is considered one of the most important factors that

affect software quality. Metrics such as cyclomatic complexity are often applied to measure fault-prone and maintainability aspects in human-written code. Nevertheless, how well this applies to AI-generated code remains an open problem. The relationship between complexity and faults in AI-generated code could be different, and the assumptions made based on human developer studies are not necessarily valid in AI-assisted programming.

This research contributes to the body of existing EBSE research by providing a large-scale empirical analysis of the security of AI-generated C code. It employs the FormAI-v2 dataset, which comprises over 883,000 AI-generated C code samples developed by nine unique AI models, all of which were tested using the Efficient SMT-Based Model Checker (ESBMC). In contrast to relying on heuristics, the research relies on formal verification outcomes, providing a more rigorous assessment of security vulnerabilities.

The research focuses on two key questions: first, whether there are differences between various AI models regarding the level of security in which they produce C code, and second, whether there is any correlation between the complexity of the code and the chances of security flaws in AI-produced code. Through the application of normalized vulnerability scores and complexity metrics, the research aims to contribute to responsible use of AI-assisted programming tools.

A. Problem and Motivation

The main concern this study seeks to address is the lack of large-scale real-world data on the reliability of AI-generated source codes in addressing security issues with different AI models. As large language models become increasingly common in today's software development process as a programming assistant, they are not just used for quick prototyping but are also used to generate production-quality codes. In this context, determining which AI models are more reliable in generating more secure codes is a critical concern.

One of the biggest challenges in evaluating the security of the code that is created by the AI is understanding the results of the vulnerability analysis. Vulnerabilities are unusual cases. Tools like static analyzers or formal verification tools can generate a lot of output that is difficult to understand unless you consider the base rates, the size of the code base, and the scope that you are analyzing. In other words, the raw count of vulnerabilities can be misleading. For example, it could imply that the security of the AI model is lower if it is producing a higher count of vulnerabilities. This is similar to the prosecutor's fallacy. The prosecutor's fallacy is a well-known problem in the literature on empirical software engineering and software testing. The fallacy occurs when the probability of finding a vulnerability is confused with the probability that a found vulnerability is a real security risk.

In addition, the big ideas of software engineering also reveal a basic truth: messy code means a greater number of bugs and a harder maintenance job. For instance, measures like cyclomatic complexity have long revealed how vulnerable a program is to bugs if it was coded by a human programmer. Yet it is not clear if the same logic is true for code created by an AI system. Programs created with AI can look very different from those created by humans with regards to how it is structured, how repetitive it is, how it utilizes control flow, and how it utilizes abstraction.

There's a clear need for comprehensive, well-supported research into the security of AI-generated code and the factors that influence it. This includes understanding the differences between the security of AI-generated code across different AI models and the relationship between the complexity of the code and the presence of security risks within AI-generated code. This research is motivated by the need to address these gaps and provide evidence to support the decision-making process regarding AI-assisted programming tools.

B. Research Questions

To address the identified problem and achieve the study's objectives, the following research questions are investigated:

RQ1: Which AI model is the safest coder in terms of security vulnerabilities? This research question examines whether different AI models produce varying numbers of security vulnerabilities when generating C code.Security is assessed by comparing the number of formally verified vulnerabilities normalized by lines of code, enabling a fair comparison across models of different output sizes.

RQ2: Does higher code complexity lead to more security vulnerabilities in AI-generated programs? This research question investigates the relationship between cyclomatic complexity and the likelihood of security vulnerabilities. It aims to determine whether more complex AI-generated programs are more prone to verification failures than simpler ones.

Together, these research questions provide a structured and evidence-based evaluation of security risks associated with AI-generated code, contributing to a more informed and responsible adoption of AI tools in software engineering practice.

II. Related Work

The security and quality of AI-generated code have become increasingly prominent research topics in recent years, driven by the rapid adoption of large language models (LLMs) for a wide range of software development tasks. AI-assisted programming tools are now commonly used to generate source code, suggest fixes, refactor existing implementations, and support developers during the software development lifecycle. As these tools become more deeply integrated into real-world development environments, concerns have emerged regarding the reliability, maintainability, and security of the code they produce.

As the use of AI-generated code increases, there are important questions being raised about its ability to meet the quality levels normally associated with software engineering. One such question revolves around the security risks associated with such code, which might be embedded during the generation process, a reality that might be realized if such code is deployed for production use cases. As such, there is an increased need for an empirical assessment of the performance

levels associated with large language models, including what they might be able to accomplish, what they might fail to accomplish, and what risks they might present.

The current research in this area is primarily divided into three categories. Firstly, the authors focus on the functional quality and correctness of the code produced by AI, which involves checking whether the code is compilable and whether it produces the correct output for a given set of tasks. Secondly, they focus on the security issues associated with code produced by AI, which involves using static analysis or automated tools to identify potential security issues in the code. Thirdly, they focus on the relationship between code complexity and software defects.

The initial research in this space was centered on code size estimation of AI-generated code using simple functional tests, such as the ability to pass a set of test cases or solve a typical programming exercise. These studies indicated that large language models are capable of dealing with simple well-defined tasks, but they did not reveal much about the quality, particularly the security-related aspects, of the code. Recently, there has been some effort to address this issue by examining typical vulnerability patterns in AI-generated code.

However, despite these advancements, there are some significant limitations of the existing literature, as the majority of the research has used limited data sets and has only focused on a single AI model, making it difficult to generalize the results. A major part of the existing literature has relied on heuristic static analysis tools, which are often associated with false positives and false negatives, making it difficult to interpret the results and leading to incorrect conclusions if the results are not considered with the appropriate context.

This research contributes to previous studies by providing a wide-ranging, security-oriented empirical study of AI-generated C code, with a basis in formal verification results. Through the FormAI-v2 dataset and an investigation of code produced by a number of different AI models, the current study provides a more thorough and extensive investigation of security vulnerabilities in AI-generated code, adding to the emerging field with new evidence and supporting the case for evidence-based evaluation of AI-assisted programming tools.

A. AI-Generated Code Quality

Several studies on the correctness and quality of code generated by LLMs have been conducted. The initial studies on GPT-3 and Codex models demonstrated the effectiveness of AI-generated code in solving a wide variety of programming problems, especially those that are well-defined and restricted in scope and scale. The studies demonstrated the potential of LLMs in assisting developers with routine coding.

The initial studies also identified some of the major challenges of AI-generated code, including logical inconsistencies, incomplete code, and a lack of robustness when dealing with complex scenarios and ambiguous code specifications. Although the code generated by the model was correct when subjected to basic tests, it was not robust enough when subjected to more complex and adverse tests.

Further studies on the code quality of AI-generated code compared the effectiveness of several AI models and the impact of prompt engineering and training data on code quality. The studies demonstrated the effectiveness of LLMs and the impact of model architecture and training data on code quality.

Although the studies demonstrated the effectiveness of LLMs and the impact of model architecture and training data on code quality, security aspects of the code generated by the models are rarely discussed. Most studies on the code quality of AI-generated code rely on test-based approaches, which may not be effective enough to test the security of the code generated by the models. Although the code generated by the model may be correct and functional, it may contain security vulnerabilities that are not identified by the test-based approach.

B. Security Vulnerabilities in AI-Generated Code

However, more recent studies have explicitly focused on the security implications of AI-generated code. Some empirical studies have shown that LLMs may produce code that contains known patterns of vulnerabilities such as buffer overflows, input validation, and the use of standard library functions. These studies have sparked concerns about the direct usage of AI-generated code in security-related applications.

Most existing studies have employed static analysis tools or heuristic vulnerability detectors to find out the security issues in software systems. Although they are useful tools in finding out the security issues in software systems, they have been observed to produce false positives and false negatives in the results they produce. Such false results may lead to reasoning errors like the prosecutor's fallacy.

In contrast, the FormAI-v2 dataset used in this study applies formal verification through the ESBMC model checker, providing stronger guarantees about detected property violations. This enables a more reliable assessment of security vulnerabilities compared to heuristic-based approaches.

- Recent research has increasingly focused on the **security implications of AI-generated code**, highlighting that large language models may produce programs containing well-known vulnerability patterns, such as buffer overflows, insufficient input validation, and unsafe use of standard library functions.
- Empirical studies have reported that these vulnerability patterns can appear even in code that is syntactically correct and functionally plausible, raising concerns about the direct deployment of AI-generated code in security-sensitive applications.
- Most existing studies rely on **static analysis tools or heuristic vulnerability detectors** to identify security issues in AI-generated software, as these tools are widely available and scalable.

C. Code Complexity and Software Defects

The relationship between code complexity and software defects is a well-researched area in traditional software engi-

neering research. Cyclomatic complexity is commonly used to measure the understandability, maintainability, and fault proneness of a program. Several empirical studies on human-developed code have shown a positive relationship between cyclomatic complexity and defect density, which implies that more complex code is generally more difficult to understand and more error-prone.

However, it is not clear if these findings are directly applicable to AI-developed code. Programs developed by large language models are likely to be quite different from human-developed code in terms of their structure, coding style, repetition, and usage of control flow constructs. Therefore, assumptions and findings obtained from empirical studies on human developers are not expected to be directly applicable to AI-developed software.

Despite the growing interest in AI-based programming, there are few studies that have empirically investigated the relationship between code complexity and AI-developed code. Even fewer studies have focused on the relationship between code complexity and security vulnerabilities in AI-developed code. This motivated us to conduct an empirical investigation to examine if existing complexity-defect relationships are applicable to AI-developed code and to what extent code complexity could be used as an indicator of security risks.

D. Positioning of This Study

The main contribution of the current research is that it extends the existing research in the field by providing a large-scale security-oriented empirical study on AI-generated C code. Unlike the previous studies in the field, the current research relies on the results obtained by formal verification tools, the normalized density of vulnerabilities, and the complexity of the code in order to evaluate the security-related properties of the AI-generated code. Such an approach enables the systematic and rigorous assessment of the security vulnerabilities.

Moreover, the current research extends the previous studies in the field by providing a joint empirical study on the influence of the AI model used and the complexity of the code on the presence of vulnerabilities in the AI-generated code. The influence of these factors on the presence of vulnerabilities in the AI-generated code is an underexplored topic in the field.

III. Research Method

The research work that is proposed in this paper intends to use a quantitative research methodology that is observational in nature to conduct empirical research on the security characteristics of C programs generated through artificial intelligence-based code generation models. Essentially, the purpose of the research methodology that is proposed in this paper is to determine how the security vulnerabilities of code generated through artificial intelligence-based code generation models vary with respect to the artificial intelligence model that is used to generate the code as well as the level of complexity in the generated code.

The research methodology that is proposed in this paper is aligned with the principles of Evidence-Based Software Engineering (EBSE), which are considered to be a set of standards that are followed while conducting research in the field of software engineering. Essentially, the research methodology that is proposed in this paper is aligned with the principles of EBSE in terms of the use of a large dataset, independent variables, dependent variables, and reproducible research procedures that are used to process the data in the research study that is proposed in this paper.

A. Dataset Description

The empirical analysis is based on the FormAI-v2 dataset, which is a significant dataset of AI-generated C programs. In total, the dataset contains 883,140 compilable C source files. These files have been generated using nine different AI models. All the files contain at least 50 lines of code. Type-1 and Type-2 code clones have already been removed from these files. Therefore, the analysis is not biased towards any particular code samples.

Each program is analyzed using the Efficient SMT-Based Model Checker (ESBMC). ESBMC is a formal verification tool that can identify any violation of predefined safety and security properties. Based on the results of the analysis, the dataset is divided into three categories. In the first category, the program verification is successful. In the second category, the program verification fails, indicating that there is some violation of properties. In the third category, the program verification is unknown. In other words, the verification of these programs is inconclusive.

Apart from the results of the formal verification of the program, some other metrics are used to analyze the characteristics of the generated code. These metrics include the number of lines of code and the cyclomatic complexity. These metrics are calculated using some of the software analysis tools.

Table I summarizes the main characteristics of the dataset and the analysis artifacts used in this study.

TABLE I
Overview of Dataset and Analysis Artifacts

Aspect	Description
Dataset	FormAI-v2
Programming Language	C
Total Programs	883,140
AI Models	9 distinct models
Minimum Program Size	50 lines of code
Verification Tool	ESBMC (formal verification)
Verification Outcomes	Successful, Failed, Unknown
Static Metrics	Lines of code, cyclomatic complexity
Clone Handling	Type-1 and Type-2 clones removed

B. Variables and Measures

To address the research questions, the following variables are used:

- Independent Variables
  - AI model: The language model used to generate each C program.

  - Cyclomatic complexity: A numeric measure of program complexity.
- Dependent Variable
  - Security vulnerability: A binary indicator representing whether a program contains at least one formally verified security violation.
- Derived Measures
  - Bug density: The number of security vulnerabilities normalized by 1,000 lines of code, used to compare AI models fairly.
  - Bug rate: The proportion of programs containing vulnerabilities within a given group (e.g., complexity category).

These measures enable consistent comparison across programs of varying size and complexity.

C. Data Preparation and Cleaning

Data analysis was carried out through Python scripting with Google Colab. Before the analysis, the data set was checked for consistency, including the presence of required data fields and data types. Since there were no inconsistencies, no data was removed during the cleaning process, and the data set used for analysis contained 883,140 samples.

Programs that were marked as *unknown* because of the verification timeout were included in the data set for completeness, but they were carefully used for further analysis. In the vulnerability analysis, only the programs with clear verification status were used, while the unknown ones were excluded from the calculations where clear vulnerability status was required. This ensures the avoidance of incorrect data while still being clear about the presence of such data.

To compare the vulnerability of different programs, regardless of their size, the vulnerability measurement was normalized with respect to the lines of code within the program. This normalizes the data to avoid bias caused by the size of the program, allowing the comparison of security performance among different AI models and program complexities to be based on relative vulnerability density rather than the actual number of vulnerabilities.

D. Analysis Procedure

The analysis was conducted in two stages, corresponding to the research questions:

- RQ1 (Safest AI Model): For each AI model, the total number of vulnerabilities was normalized by the total number of lines of code generated by that model. This resulted in a bug density metric (bugs per 1,000 lines of code), enabling direct comparison of security performance across models.
- RQ2 (Complexity and Vulnerabilities): Programs were grouped into complexity categories based on cyclomatic complexity values. Bug rates were calculated for each category, and statistical correlation analysis was performed to evaluate the relationship between complexity and vulnerability presence.

To assess statistical significance, correlation coefficients and corresponding p-values were computed. Visualization techniques, including bar charts and scatter plots, were used to support interpretation of the results.

E. Validity and Reproducibility Considerations

All the steps in the analysis were performed using reproducible scripts. Additionally, the data set used is publicly available. Finally, the use of formal verification results as opposed to heuristic detection methods improves the internal validity of the results. However, the study is observational in nature and does not purport to establish any causal effects. The potential limitations with regards to tool limitations, data set composition, and model imbalance are discussed in the section on Threats to Validity.

IV. Results

In this section, the empirical results from the security analysis performed on the FormAI-v2 dataset will be provided. The results will be presented based on the two research questions. In addition, visualizations will be used to make the results easier to understand.

A. Dataset-Level Overview

The analysis includes 883,140 C programs that were generated by nine different models of AI. Out of the total number of C programs, 799,335 were found to be vulnerable. Therefore, the percentage of vulnerable C programs was found to be 90.51%.

The percentage of vulnerability for different models of AI is shown in Fig. 1. From the figure, it is observed that all models of AI have shown high percentages of vulnerable files.

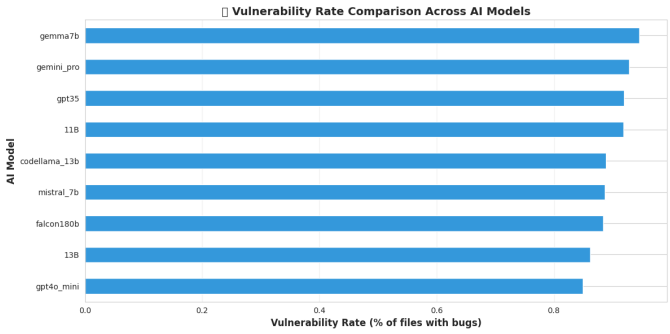


Fig. 1. Vulnerability rate (% of files with at least one security bug) across AI models.

B. RQ1: Which AI Model Is the Safest Coder?

To answer RQ1, the measure for vulnerability density was determined by the number of security bugs for every 1,000 lines of code for each AI model. This approach helps in the fair comparison of different models that produce different amounts and sizes of code.

Fig. 2 shows the bug density for each AI model, ranging from the least to the most. The findings reveal a clear distinction between the models:

- GPT-4o-mini has the lowest bug density, which implies relatively safer code generation.
- GPT-3.5 and Gemini Pro come next with a slightly higher bug density.
- Models like CodeLlama-13B, Falcon-11B, Falcon-13B, and Mistral-7B come under the middle category.
- Gemma-7B has the highest bug density, which implies the highest relative security risk.

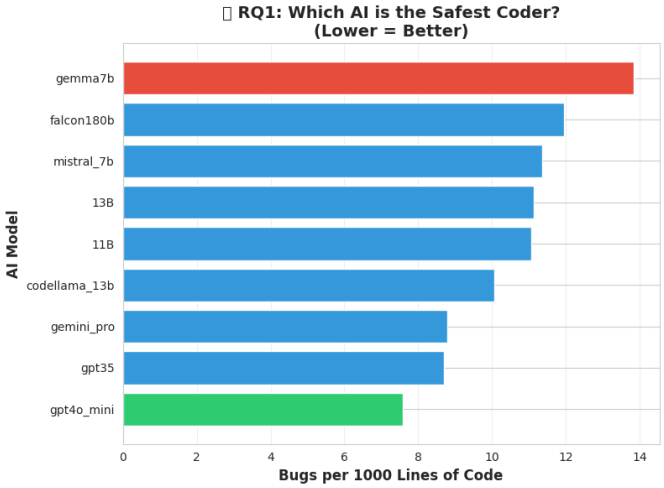


Fig. 2. Security bug density (bugs per 1,000 lines of code) for each AI model.

To provide a complement to this normalized understanding, Fig. 3 illustrates the total number of vulnerabilities identified per model. Although models with greater output sizes are expected to have greater absolute numbers of vulnerabilities, this graph illustrates the problem with simply looking at absolute numbers for an understanding of security quality and the need for normalized values. The data clearly illustrates that AI models are not security equivalent.

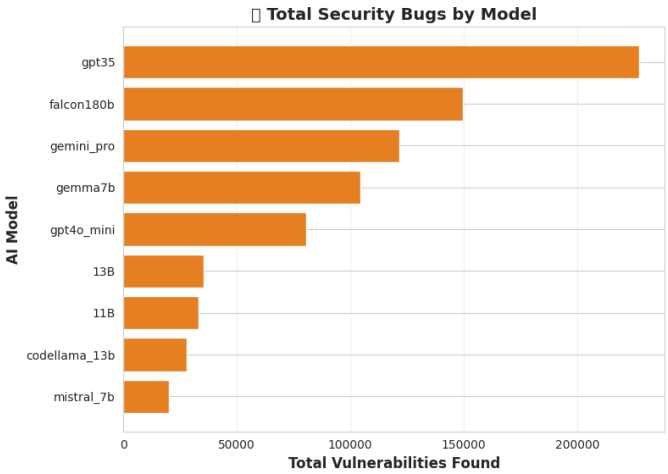


Fig. 3. Total number of security vulnerabilities detected per AI model.

C. RQ2: Does Code Complexity Lead to More Security Bugs?

RQ2 investigates the relationship between code complexity and security vulnerabilities in C programs generated by AI. For the purpose of this analysis, the C programs were grouped into three categories based on their cyclomatic complexity measures: Simple `0 <= CC <= 2`, Moderate to High `2 < CC <= 10`, and Very Complex `CC > 10`. This allows us to understand the frequency of vulnerabilities for different levels of complexity.

As depicted in Figure 4, the vulnerability rates vary from one defined complexity level to another. It was surprisingly found that the bug rate was at its highest in Simple programs, and the lowest was recorded for Moderate to High Complexity, where `5 <= CC <= 10`. The bug rates for Very Complex programs were slightly higher than those for Moderate to High Complexity, though lower than those for simple programs.

These results indicate that in our data set, there is a tendency for more cyclomatic complexity to be associated with a slightly lower probability of security bugs in the source code generated by AI. In other words, it appears simpler programs may be more likely to be vulnerable to bugs. Although the actual difference in bug probabilities between complexity categories is not large, this pattern holds across the data set and is again represented in Figure 4. This demonstrates that cyclomatic complexity alone is not necessarily useful in determining security risk in AI-generated code.

The above observations imply that the occurrence of vulnerabilities is not significantly related to cyclomatic complexity. Rather, complexity is likely to have a very limited effect on the likelihood of vulnerabilities, as seen in the modest differences in bug rates between levels. Although the differences are not large in magnitude, they are systematic rather than random, and thus, complexity alone cannot be considered a reliable predictor of security vulnerability in the examined AI-generated programs.

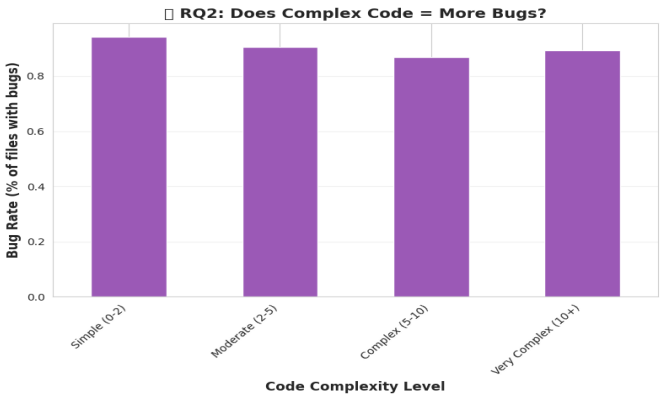


Fig. 4. Security bug rate across cyclomatic complexity categories.

To further investigate this relationship, a scatter plot between cyclomatic complexity and vulnerability presence with a trend line has been provided in Fig. 5. The scatter plot indicates a weak downward trend line, which implies that there

is no positive correlation between increased complexity and vulnerability presence. A statistical correlation analysis has been conducted on this data, which has resulted in a correlation coefficient of -0.0437 with a p-value <0.05.

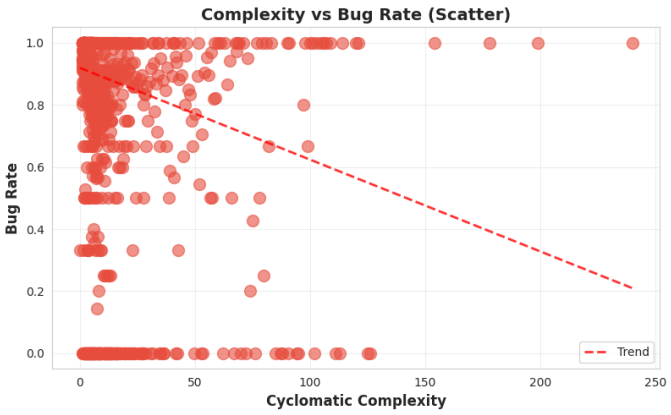


Fig. 5. Relationship between cyclomatic complexity and vulnerability presence with fitted trend line.

D. Vulnerability Characteristics

Apart from the overall vulnerability rates, the dataset was also used to determine the common patterns of vulnerability.Fig. 6 shows the top 15 security vulnerabilities that commonly exist in the code. From the distribution of the top 15 security vulnerabilities, it is clear that the security vulnerabilities that exist in the AI-generated code have a common programming pattern.

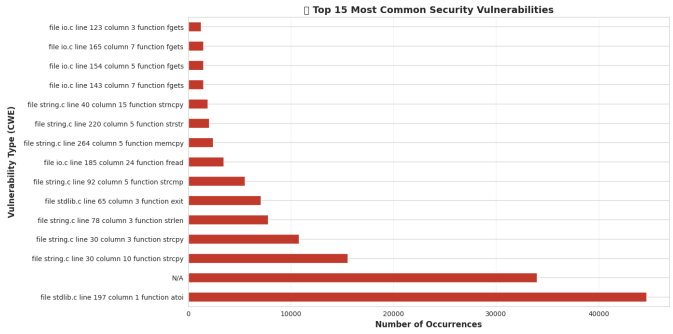


Fig. 6. Top 15 most frequently occurring security vulnerabilities in the FormAI-v2 dataset.

E. Summary of Results

The key findings from the results can be summarized as follows:

- AI models differ substantially in vulnerability density when normalized by lines of code.
- GPT-4o-mini exhibits the lowest bug density, while Gemma-7B exhibits the highest.
- Code complexity does not positively correlate with vulnerability likelihood; instead, a weak but statistically significant negative relationship is observed.
- Common vulnerabilities are dominated by recurring unsafe programming patterns.

Table II consolidates the main empirical findings of this study by summarizing the results related to the security of AI models, code complexity, and recurring patterns of vulnerability. The table indicates that there is a significant variation in the security performance of different AI models when the density of vulnerability is normalized by lines of code. Furthermore, it indicates that the complexity of cyclomatic code has a weak association with the occurrence of vulnerability, which again confirms that complexity is not a significant determinant of security risk. Also, the table indicates that many detected vulnerabilities occur due to recurring programming patterns.

TABLE II
SUMMARY OF EMPIRICAL RESULTS FROM FORMAI-V2 ANALYSIS

Dimension	Observed Result
AI Model Security (RQ1)	Vulnerability density varies across models. GPT-4o-mini records the lowest rate (7.58 bugs per 1,000 LOC), whereas Gemma-7B records the highest (13.84 bugs per 1,000 LOC), indicating notable differences in security performance.
Code Complexity (RQ2)	A weak but statistically significant negative association is observed between cyclomatic complexity and vulnerability presence (correlation coefficient = -0.0437, $p<0.05$), suggesting that higher complexity does not increase vulnerability likelihood.
Vulnerability Characteristics	Security issues are predominantly concentrated in a limited set of recurring vulnerability types, with memory-related and unsafe function usage patterns appearing most frequently among detected violations.

These results provide the empirical basis for the interpretation and implications discussed in the next section.

V. DISCUSSION

In this section, the implications of the empirical findings presented in Section 4 are discussed. Emphasis is given to their implications on the domain of AI-assisted software development. In addition, the limitations of the study are taken into consideration in order to avoid potential threats to the validity of the results. The discussion is structured in a manner that prevents the drawing of generalized conclusions.

A. Implications of the Findings

1) Implications for AI-Assisted Software Development: The implications of the answers to RQ1 are that the quality of the code's security developed by AI is highly variable. From the number of vulnerabilities that appear, it is clear that not all AI models are equal in terms of their reliability. Therefore, the choice of AI model used has significant implications for the security of the code developed.

Selecting an AI model with lower vulnerability density can be beneficial in reducing the overall number of security issues

that emerge during the code generation process. However, the point that all models have a consistently high rate of vulnerabilities indicates that AI code is not secure by default. Even the better models have code with a non-negligible number of vulnerabilities.

The results suggest that code generated by AI systems should be treated as a starting point or a draft rather than a final solution or answer. When you introduce code generated by AI systems into a software system, you should use conventional security practices like manual code review, extensive testing, and automated security analysis. Although AI-based coding tools can help developers be more efficient in coding, they do not replace traditional software development practices that make code secure and reliable.

2) Implications for Security Assessment and Tool Interpretation: As such, the study reveals a number of vulnerabilities, which also emphasizes the need for us to be careful with the output of such security tools. As such, even though tools such as static analysis tools and formal verification tools help us identify potential security problems, such problems might be misinterpreted, which, as the course materials emphasize, might lead to the prosecutor's fallacy, whereby even if a vulnerability exists, this does not necessarily imply that the code is highly insecure.

To mitigate this issue, the paper examines security outcomes from the perspective of vulnerability density, measured in terms of bugs per 1,000 lines of code, rather than simply focusing on the number of vulnerabilities. This normalization helps to account for differences in the size of the code generated by different AI models and prevents models that generate larger amounts of code from being considered less secure simply because they have more vulnerabilities.

From the results obtained, it is evident that comparing data in relative or normalized terms is more beneficial in understanding the security characteristics of AI-generated codes compared to comparing absolute numbers. This underscores the need to interpret the results obtained from automated security analysis with caution and to prefer normalized numbers when comparing different tools or models with varying sizes.

3) Implications of the Complexity Analysis: The result of RQ2 indicates a small but statistically significant negative relationship between cyclomatic complexity and security vulnerabilities. This is contrary to the general software engineering belief that more complex code is likely to have more bugs. However, in the case of AI-generated code, it appears that more complexity does not necessarily mean more security risk.

One possible explanation for this phenomenon is the fact that simpler AI-generated programs may rely more on unsecure programming patterns or lack control flow analysis, making them more vulnerable to particular types of violations. More intricate programs may include additional conditional logic and structural controls, making them less vulnerable to particular types of violations. Another possible explanation for this phenomenon is the behavior of formal verification tools, which may be able to analyze simpler programs more effectively. Violations of simpler code may be more likely to be detected, while the increased intricacy of more complex code may cause the tools to reach their limits and detect fewer violations.

Whatever the reason for it, the above studies indicate that cyclomatic complexity, in and of itself, is not a reliable predictor of security vulnerabilities in code produced by AI. This is a relevant finding because, as a general rule, the assumptions underlying the relationship between complexity and defect proneness have been based primarily on studies of code produced by humans. Hence, the evaluation criteria we generally apply to software produced by humans may not necessarily apply to software produced by AI and needs to be reevaluated in this context.

B. Limitations and Threats to Validity

Despite the large scale of the dataset and the use of formal verification, this study is subject to several limitations and threats to validity.

1) Internal Validity: The paper relies on vulnerability labels provided by the ESBMC model checker. Although formal verification methods offer a higher level of assurance than static analysis methods, there are limitations in the properties checked, constraints applied, and timeout limits. Programs classified as unknown may contain vulnerabilities or be safe, which raises questions about the reliability of the approach.

2) Construct Validity: Security vulnerability is treated as a binary variable indicating the presence of at least one property violation. This approach does not account for:

- Severity of vulnerabilities
- Number of vulnerabilities per program
- Exploitability in real-world settings

Similarly, cyclomatic complexity is used as a proxy for program complexity. While widely accepted, it does not capture all aspects of code structure or readability, particularly in AI-generated code.

3) External Validity: The dataset consists exclusively of AI-generated C programs, and the findings may not generalize to:

- Other programming languages
- Human-written code
- Different AI models or future model versions

Additionally, the tasks used to generate the programs may not fully represent real-world software development scenarios.

4) Conclusion Validity: The large sample size in our study provides us with a powerful statistical test, which makes it easier to detect even small effects as statistically significant. This increases the validity of our statistical tests, but it also means that we have to be cautious in our interpretation of the results. In conclusion, the results that are statistically significant may actually represent small effects that are not necessarily important.

This is evidenced by the results obtained in RQ2, where there is a statistically significant relationship between cyclomatic complexity and the presence of vulnerabilities. However, the results indicate that the relationship is low, implying that code complexity explains a small percentage of the variation in the results obtained in terms of vulnerabilities.

Since this is an observational study, causation cannot be established. It is a study that points to relationships between variables but doesn't establish that changes in code complexity directly influence security vulnerability. Thus, a conservative approach is taken in drawing conclusions, and the study is termed descriptive and correlative rather than causal. This is a best practice in evidence-based software engineering and helps the reader understand the results in a balanced light.

VI. Conclusions

This research undertakes a comprehensive empirical analysis of the security implications of C code generated using AI, based on the FormAI-v2 dataset. The dataset consists of 883,140 verifiable C code compiled using nine different AI models. The primary aim of the research is to provide evidence-informed insights into the security implications associated with the use of AI in code development and how these implications vary depending on the AI model and code complexity.

From the data, it can be seen that there is a significant difference in the security level of the code that can be produced by different AI models. If you consider the number of vulnerabilities produced per line of code, you can notice differences in vulnerability density between different models. It can be said that AI models are not the same when it comes to security, and the security level may depend on the specific tool used.

Furthermore, the code complexity analysis revealed some findings which do not conform to conventional software engineering intuitions. For example, there was a weak yet statistically significant negative relationship between cyclomatic complexity and the presence of vulnerabilities. In other words, for AI-generated C code, more complexity is not necessarily associated with more security risk. Although this relationship is weak, it challenges conventional intuitions based on studies of human-written code and suggests that conventional complexity-based intuitions may not be directly applicable to AI-generated software.

This research contributes to the existing evidence base in evidence-based software engineering by providing large-scale empirical findings grounded in the results of formal verification. It is evident from this research that code developed by AI cannot be considered secure by default, and this highlights the continued need for security analysis, thorough testing, and code review when this code is integrated with software systems. The results of this research also highlight the need for selecting the correct AI model and the correct evaluation metric when assessing the security of code developed by AI. Future research directions could include the extension of this research by considering more programming languages, types of vulnerabilities, and newer AI models to gain further empirical insights into AI-assisted software development.

Acknowledgments

We would like to thank the course instructor for providing guidance and feedback throughout the course. The FormAI-v2 dataset used in this study is gratefully acknowledged. This work was conducted as part of an academic assignment, and no external funding was received.

References

[1] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.
[2] E. Nijkamp et al., "Code Generation with Large Language Models: A Survey," *ACM Computing Surveys*, 2023.
[3] Y. Li et al., "An Empirical Study of Code Generation with Large Language Models," in *Proc. ICSE*, 2022.
[4] H. Pearce et al., "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *Proc. IEEE Symposium on Security and Privacy*, 2022.
[5] O. Asare et al., "Evaluating the Security of AI-Generated Code," in *Proc. ESEM*, 2022.
[6] Y. Zhou et al., "On the Security Risks of Large Language Models Generating Code," *arXiv preprint arXiv:2302.XXXX*, 2023.
[7] L. C. Cordeiro et al., "SMT-Based Bounded Model Checking for Embedded ANSI-C Software," *IEEE Transactions on Software Engineering*, vol. 38, no. 4, 2012.
[8] E. Clarke, T. Henzinger, and H. Veith, *Handbook of Model Checking*. Springer, 2018.
[9] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, 1976.
[10] N. Fenton and M. Neil, "A Critique of Software Defect Prediction Models," *IEEE Transactions on Software Engineering*, 1999.
[11] V. R. Basili et al., "The Empirical Investigation of Perspective-Based Reading," *Empirical Software Engineering*, 1996.
[12] B. Kitchenham et al., "Evidence-Based Software Engineering," *Proc. ICSE*, 2004.
[13] G. Gigerenzer, *Reckoning with Risk*. Penguin, 2002.